TYPE SCRIPT

→ Typescript is an open source object-oriented language developed and maintained by microsoft.

→ Typescript is a superset of javascript that compiles to plain javascript.

→ Basically, Typescript is the ES6 version of javascript with some additional features.

→ official website: https://www.typescriptlang.org

why Typescript?

→ Before typescript & javascript is well established in the market.

→ using JS we can develop both client and server side applns using different frameworks like Angular or React.js and Node.js

(Angular or React.js: client side appln frameworks; Node.js: server side appln framework)

→ Javascript is the dynamically programming language with no typesystem.

→ we need a language with type system which increases code quality, readability and makes it an easy to maintain and refactor code base.

→ More importantly, errors can be identified at the compile time rather than at run time.

→ So that without the type system, it is difficult to scale JS to build complex applications. Hence, the reason to use Typescript is that it support type system and allows JS to be used at scale to build complex applications.

How to use Type Script?

→ A Typescript code is written in a file with .ts extension and then compiled into javascript using the Typescript compiler

→ A Typescript file can be written in any code editor.

→ A Typescript compiler needs to be installed on your platform. Once installed, the command

tsc <filename>.ts

compiles the Typescript code into a plain JS file

→ Javascript files can then be included in the HTML and run on any browser

Sample.ts —tsc Sample.ts→ Sample.js

Features of TypeScript:

1) Cross platform: The typescript compiler can be installed on any OS such as windows, MacOS and Linux.

2) Object-oriented Language: Typescript provides features like classes, interfaces, and Modules.

3) Static type-checking: Typescript uses static typing and helps type checking at compile time. Thus, you can find errors while writing the code without running the script.

4) Optional Static Typing: Typescript also allows optional static typing in case you are using the dynamic typing of ~~Stri~~ Javascript.

5) DOM Manipulation: You can typescript to manipulate the DOM for adding or removing elements.

6) ES 6 features: Typescript includes most features of planned ECMA Script 2015 (ES6,7) such as Class, interface, Arrow functions etc.,

Advantages:

→ Open-source language → runs on any stat browser

→ Similar to Javascript and the same syntax and semantics.

→ Syntax to Closer to back-end languages.

→ TypeScript support for the latest JS features.

Summery of TypeScript:

→ Object oriented programming language & Superset of JS.

→ Typescript code needs to be compiled

→ There is static typing. we can declare variables with data types.

Ex: var num: number

→ It has interface

→ It has optional parameter feature

→ It has Rest parameter feature.

→ Generics supported

→ Modules supported

→ Number, string are interfaces.

Variables:

→ Variable is a container which can hold data
→ Typescript follows some rules as JS for variable declarations.
→ JS is not a typed language. It means we cannot specify the type of variable such as number, string, boolean etc.,
→ However, Typescript is a typed language, where we can specify of the variables.
→ variables can be declared using: var, let and const.
* we can declare the variables in four different ways.
1) Both type and initial value 3) only the initial value
2) only the type 4) without type and intial value.

var:

1) `var employee_name : String = "John";`
   - var → key word
   - employee_name → variable name
   - String → type
   - "John" → value of string

   `console.log(employee_name);` → John

2) `var employee_name : string;` → declaration (Type)
   `employee_name = "John";` → Initialization

3) `var employeeName = "John";`

4) `var employeeName;`
   `employeeName = "John"`

var:

→ The variables declared using var are available with in the function.
→ If we declare them outside the function, then they are available everywhere i.e., they are a global variable.
→ If we define them inside a code block, they are still scoped to the enclosing function.

Ex:

```
function SomeFn()
{
    if (true)
    {
        var localvar = 1000;
        console.log(localvar) // OK
    }
    console.log(localvar); // OK
}
Console.log(localvar) // error
```

Scope of variable

Local Scope var:

Global scope var:

```
var x=100
function somefn()
{
    if(true)
    {
        var x=1000;
        console.log(x);
        console.log(y);
    }
    console.log(x);
}
console.log(x); // error
console.log(y);
```

Scope of x will be within { }

x is local
y is global

Let:

The variables declared using let can only use it in the code block where we declare them. Outside the code block, they are invisible.

→ If they are outside the code block, but within function body then they become function scoped.

Local Let:

```
function somefn()
{
    if (true)
    {
        let localvar=1000;
        console.log(localvar); // Ok
    }
    console.log(localvar); // error
}
console.log(localvar); // error
```

Scope of let localvar

Global Let

```
let y=100;
function somefn()
{
    if(true)
    {
        let x=200;
        console.log(x); //Ok
        console.log(y); //Ok
    }
    console.log(x); // error
}
console.log(x); // error
console.log(y); //Ok
```

y Scope

x Scope

* var - Scope is within function
* let - Scope is within block
* -> we can update or redefine a var variable.
* -> The let keyword cannot be redeclared within the scope but can be updated (reassigned a new value).
* -> The const variables cannot be redeclared or updated within the same scope.

ex: Redefining of var variable:

1) var x = 100;
   console.log(x);
   var x = 200;
   console.log(x);

valid

o/p: 100
     200

tsc ./variables.ts ← compile
node ./variables.js ← run

2) var x = 100;
   console.log(x);
   var x: string = "John";
   console.log(x);

o/p
100
John

← compilation error will throw
← Runs successfully

Const:
-> we must declare a const variable with an initial value.
-> The value cannot be reassigned again.

TYPESCRICT DATATYPES:

- Built-in (primitive)
  - Number
  - void
  - string
  - Null
  - Boolean
- Userdefined (non-primitive)
  - Array
  - Class
  - Tuple
  - Enum
  - Interface
  - function

Primitive (Built in Datatypes):

1) Number:

```
var first: number = 12.0;
var second: number = 0x37CF; -> Octa number
var third: number = 0o377; -> Hexadecimal number
var fourth: number = 0b1111001; -> Binary number.
console.log(first);
console.log(second);
console.log(third);
console.log(fourth);
```

tsc ./Numberfile.ts ← compile
node ./Namefile.js ←

o/p:
12
14287
255
57

} Decimal values

2) String:

```
var empName : string = "John";
var empDept : string = "IT";
console.log(empName); // John
console.log(empDept); // IT
var stmt : string = empName + " works in " + empDept;
console.log(stmt); // John works in IT
```

3) Bodean:

```
var b : bodean = true;
console.log(b); //true
```

4) void:

```
function hello() : void {
    console.log("This is welcome message."); // This is welcome message.
}
```

5) Null: Null represents a varrable whose value is undefined.

```
var num1 : number = null
num1 = 100;
console.log(num1); //100
var num2 : number = undefined;
num2 = 100;
console.log(num2) //100
```

undefined: undefined primitive type denotes all uninitialized varicables.

Anytype: Accepts all types of data.

```
var val : any = "Hi";
console.log(val); // Hi
val = 100;
console.log(val); // 100
val = false
console.log(val); // false.
```

Ex:

```
function myfunction (x : any, y : any) {
    console.log(x + y);
}
myfunction(100, 200); // 300
myfunction("Hi", " welcome"); // Hi welcome
```

TYPESCRIPT OPERATORS

→ Operator is a symbol which will perform a train operation

→ For example, if we take simple expression 4+5=9

Here 4 and 5 are Operands

'+' is called operator

→ Typescript consists of different types of operators that are used to perform different operations.

Types of JS operators:

1) Arithmetic operators 2) Assignment operators

3) Relational / comparison operators 4) Logical operators

1) Arithmetic operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
* *	Exponentiation
/	Division
%	Modulus (Division Remainder)
+ +	Increment
- -	Decrement

2) Assignment Operator

Operator	Example	Same as
=	x = y	x = y
+=	x += y	x = x + y
-=	x -= y	x = x - y
*=	x *= y	x = x * y
/=	x /= y	x = x / y
%=	x %= y	x = x % y

Example:

Arithmetic operators

```
var x=10;
var y=20;
console.log(x+y); //30
console.log(y-x); //10
console.log(x*y); //200
console.log(x/y); //0.5
console.log(x%y); //10
console.log(5**2); //25
x++;
console.log(x); //11
y--;
console.log(y); //19
```

Assignment operators.

```
x=100;
y=50;
x=x+y;
console.log(x); //150
x+=y; => x=x+y //150
console.log(x-=y); => x=x-y => 100
console.log(x*=y); => x=x*y => 5000
console.log(x/=y); => x=x/y => 100
console.log(x%=y); => x=x%y => 0
```

3) Relational operators: >, <, >=, <=, !=

4) Logical operators: And, OR, NOT

Example:

```
x=10;
y=20;
console.log(x>y); // false
console.log(x<y); // True
console.log(x>=y); // false
console.log(x<=y); // true
console.log(x!=y); // True
```

5) Ternary Operator:

```
console.log(x>y?x:y); //10
console.log(x>y?x:y); //20
```

x → greater returns x
y → greater returns y

```
var a: boolean = true;
var b: boolean = false;
console.log(a && b); // false
console.log(a || b); // true
console.log(!a); // false
```

AND — Both should be true

OR — Any one should be true

NOT — Viceversa (T → F, F → T)

TYSCRIPT STRINGS

Declaration:

1) `var mystring: string = "Hello"; "welcome to typescript";`

2) `let mystring1 = "world"; "welcome";`

Methods:

1) charAt()

```
Console.log(mystring.charAt(0)); // w
console.log(mystring.charAt(2)); // l
```

2) concat()

```
var str1: string = "welcome";
var str2: string = " To typescript");
console.log(str1.concat(str2)); // welcome To typescript
```

3) Replace()

```
var str = "Typescript programming";
console.log(str.replace('i', 'x')); // Typescxpt programming
console.log(str.replace("Type", "Java")); // Java Script programming
```

← only first character will be replaced.

4) split()

```
var fruits: string = "Apple Banana Orange";
console.log(fruits.split(' ')); // ['Apple', 'Banana', 'Orange']
console.log(fruits.split(' ', 2)); // ['Apple', 'Banana']
console.log(fruits.split(' ', 1)); // ['Apple']
```

5) Substring()

```
var str = "welcome";
Console.log(str.substring(0, 3)); // wel
                                  // 3-1 = 2
console.log(str.substring(2, 5)); // lco
                                  // 5-1 = 4
```

Index 1 2 3 4 5 6 7
welcome
position 0 1 2 3 4 5 6

6) toUpperCase() && toLowerCase()

```
var Str = "Typescript programming";
Console.log( str.toLowerCase()); // typescript programming
Console.log( Str.toUpperCase()); // TYPESCRIPT PROGRAMMING
```

7) trim()

```
Str = "      welcome      ";

Console.log (Str.trim()); // welcome
Console.log (Str.trimRight()); // ___ welcome
Console.log( str.trimLeft()); // welcome ___
```

TYPESCRIPT CONDITIONAL STMTS

1) If 2) Ifelse 3) Elseif 4) Turnery 5) Switch.

1) If & if else

```
var person_age : number =20;
    if (person_age >= 18)
    {
      console.log("Eligible for vote");
    }
    else
    {
      console.log(" NOT eligible for vote");
    }
```

2) Else if

```
var a:number =10;
var b:number =20;
var c: number=30;
if (a>b && a>c)
   { console.log("A is greater");
   }
else if (b>a && b>c)
   { console.log (" B is greater");
   }
else {
      console.log( "C is greater");
```

4) Turnery operator

```
var x : number =100
var y : number =200
x>y ? console.log ("X is largest"): console.log ("y is greater");
```

5) Switch

```
var weekno: number = 1;
switch (weekno)
{
  case 1: console.log("sunday"); break;
  case 2: console.log("Monday"); break;
  case 3: console.log("Tuesday"); break;
  case 4: console.log("wednesday"); break;
  case 5: console.log("Thursday"); break;
  case 6: console.log("friday"); break;
  default: console.log("Invalid weekno.");
}
```

TYPE SCRIPT LOOPS

* Loops are a set of stmts which will be executed for multiple times.

1). while 2) DO while 3) For

* Jumping stmts with loops 1) Break 2) continue

1) while:

```
var i:number = 1;
while(i <= 10)
{
  console.log(i); // 1 ... 10
  i++;
}
```

2) DO while: Even the condition is not matching atleast once execution will be done.

```
do
{
}
while(condition)
```

```
var i:number = 20;
do
{
  console.log(i); //20
  i++;
}
while(i<=10)
```

condition fails.

3) For Loop:

```
For(var i:number = 1; i<=10; i++)
{
  console.log(i); // 1 ... 10
}
```

1) Break: used for jumping from the loop.

```
for(var i:number = 1; i<=10; i++)
{
  if(i==5)
  {
    break; -> Quits loop at i=5
  }
  console.log(i); // 1,2,3,4
}
```

2) Continue:

```
for (var i:number=1; i<=10; i++)
{
    if(i==5)
      { continue; } → 5 will be skipped
    console.log(i); // 1,2,3,4,6,7,8,9,20.
}
```

TYPESCRIPT ARRAYS:

An array is a special type of datatype which can store multiple values of different datatypes sequentially using a special syntax.

→ There are 2 types of arrays.

1) Single dimentional
2) Two dimensional array.

1) Single Dimensional Array:

Single Type Array

```
1) var fruits:string[] = ["Apple", "Banana", "mango"]
```

(or)

```
2) var fruits2: Array<string>;
   fruits2 = ["Apple", "Mango", "Banana"]

   console.log(fruits); // ["Apple", "Banana", "Mango"]
   console.log(fruits2); // ["Apple", "Mango", "Banana"]
```

Multi Type Array

```
1) var values:(string | number)[] =
        ["Apple",100,"Orange",10];
```

(or)

```
2) var values2: Array<string | number> =
        ["RAM",10,20,"Anjali,"];
   console.log(values);
   console.log(values2);
```

Array Accessing Elements

```
var fruits:string[] = ["Apple", "Mango"];
   console.log(fruits[0]); // Apple
   console.log(fruits[1]); // Mango
   console.log(fruits[2]); // Undefined
```

For loop:

```
for(var i=0; i<fruits.length; i++)
{ console.log(fruits[i]);
}
```

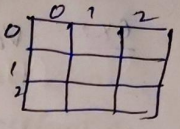
Foreach

```
for(var j in fruits)
{ console.log(fruits[j]);
}
```

2) Two dimensional Array:

Declaration & Initialization

```
var myarray: number[][] = [[10,20] [30,40]];
```




```
console.log(myarray);
```

(or)

```
var myarray2: (string|number)[][] = [[10, xyz], ["abc", 20]];
                                     0,0  0,1    1,0  1,1
console.log(myarray2);
```

Array elements Accessing:

```
console.log(myarray2[0][0]);
Console.log(myarray2[0][1]);
console.log(myarray2[1][0]);
Console.log(myarray2[1][1]);
```

For Loop:

```
for(var i=0; i<myarray2.length; i++)
{
  for(var j=0; j<myarray2[i].length; j++)
  { console.log(myarray2[i][j]);
  }
}
```

In operator

```
for(var i in myarray2) //row
{
  for(var j in myarray2[i]) //
  {
    console.log(myarray2[i][j]);
  }
}
```

TUPLE IN TYPESCRIPT

* Tuple is a new data type which includes multiple set of values of different data types.
* It represents a heterogeneous collection of values.
* Tuple type variable:

Ex: `var employee = [1, "steve"];`

(OR)

`var employee: [number, string] = [1, "steve"];`

Ex:

```
var employee: [number, string, number] = [101, "John", 5000];
console.log(employee);
Console.log(employee[0]);
Console.log(employee[1]);
Console.log(employee[2]);
```

Add elements in to Tuple

```
var employee: [number, string, number] = [101, "John", 5000];
console.log("Original values of tuple:" +employee); // 101 John 5000
employee.add(102, "scott", 7000);
Console.log("After adding new elements:" +employee); //
```

101 John 5000 102 Scott 7000

Remove elements from tuple

```
employee.pop();
Console.log("After removing elements:" +employee); //
```

Update elements in tuple

101 John 5000
102 Scott

```
var student: [number, string] = [101, "smith"];
student[1] = "scott";
console.log(student); // 101 scott
```

Tuple Array

```
var student: [number, string];
student = [[101, "smith"], [102, "John"], [103, "Jimmy"]];
console.log(student[0]); // [101, 'smith']
console.log(student[1]); // [102, 'John']
console.log(student[2]); // [103, 'Jimmy']
```

Functions in Typescript

* Functions are primary blocks of any program.
* Advantages:

1) Code reusability: we can call a function several times without writing the same block of code again.
The code reusability saves time and reduces the program size.

2) Less coding: Functions makes our program compact. So, we don't need to write many lines of each code each time to perform a common task.

3) Easy to Coding: It makes the programmer easy to locate and isolate faulty information.

Types of functions:

1) Named Function:
A named function is one where you declare and call a function by its given name.

2) Anonymous function:
* An anonymous function is one which is defined as an expression.
* This expression is stored in a variable. So, the function itself does not have a name.
* These functions are invoked using the variable name that the function is stored in.

Named f( ):

Ex:1
```
function display( )        → F'( ) Name
{
    console.log("welcome to typescript");
}
display( );  → F'( ) Call.
```

Ex:2
```
function sum (x:number, y:number) : number      → M'( ) Name, → Return type
{
    return x+y;
}
console.log ( sum(5+10));      → M'( ) Call
```

Anonymous f'l():

Ex:
```
var greeting = function()
{
    console.log("welcome to TypeScript");
}
greeting();
```

Ex: Anonymous f'l() includes parameter types and return type.

```
var Sum = function(x:number, y:number): number
{
    return x+y;
}
Sum(40,5);
```

(number → Return type; x, y → parameters)

Function parameters:

* parameters are values or arguments passed to a function.
* In TypeScript, the compiler expects a function to receive the exact number and type of arguments as defined in the function signature.
* If the function expects three parameters, the compiler checks that the user has passed values for all three parameters i.e., it checks for exact matches.
* Types of parameters:
  1) Optional parameters
  2) Default parameters

```
function Greet(greeting:string, name:string):string
{
    return greeting+" "+name;
}
console.log(Greet("welcome","John");
console.log(Greet("welcome");  // compiler error 1 parameter only passed
console.log(Greet("welcome","John","smith");  // compiler error 3 parameters passed
```

optional parameter

→ optional parameter

```
function Greet(greeting: string, name?: string): string
{
    return greeting + " " + name;
}
console.log("welcome", "John");
Console.log(
Console.log(Greet("welcome", "John"));
Console.log(Greet("welcome")); // 2nd arg will be undefined  OK
Console.log(Greet("welcome", "John", "Smith")); // Error since 3 parameters passing.
```

Default parameters => If we wont pass any argument then it will take the default value.

```
function Greet(name: string, greeting: string = "Hello"): string
{
    return greeting + " " + name;
}
Console.log(Greet('John')); // OK returns Hello John
Console.log(Greet('John', 'welcome')); // Hello John
Console.log(Greet('smith')); // Hello smith
```

ARROW FUNCTION:

→ Fat arrow notations are used for anonymous function i.e., for function expressions.

→ They are also called lamda functions in other languages.

Syntax:

```
(param1, param2, ..., paramN) => expression
```

Ex:

```
var sum = (x: number, y: number): number =>
{
    return x + y;
}
console.log(sum(20, 30));
```

Ex 2:

```
var print = () =>
{
    console.log("welcome to typescript");
}
print();
```

(or)

```
var print = () => console.log("welcome to typescript");
print();
```

Note: If the function body consists of only one statement then no need for curly brackets and the return keyword.

```
Var sum = (x:number, y:number): number =>
{
    return x+y;
}
```

(or)

```
var sum = (x:number,
    y:number) => x+y;
```

```
console.log(sum(10,5));
```

Function Overloading:

* Typescript provides the concept of function overloading.
* we can have multiple functions with the same name but different parameter types and return type. However, the number of parameters should be the same.

Ex:

```
function add(x:number, y:number):number
function add(x:string, y:string): string

function add(a:any, b:any) : any
{
    return a+b;
}
console.log(add(1,2));
console.log(add("welcome", "John"));
```

** Function overloading with different number of parameters and types with same name is not supported.

Ex:

```
function display(a:string, b:string) :void
{
    console.log(a+b);
}
function display(a:number): void
{
    console.log(a+b);
}
```

(a:string, b:string) — Return types vary

(a:number) — no. of arg vary

↳ Compile time error

Typescript Executor:

⇒ npm install -g tsx

Rest parameters:

* If number of parameters that a function will receive is not known or can vary, we can use rest parameters.
* we can use rest parameter denoted by ellipsis ...

Ex:

```
function Greet(greeting: string, ...name: string[])
{
    return greeting + " " + name;
}
Console.log(Greet("Hello", "John")); // Hello John
Console.log(Greet("Hello")); // Hello
Console.log(Greet("Hellow", "John", "Smith")); // Hello John Smith
```

Class & Object in Typescript

Class:

A class can include the following 1) Constructor 2) properties 3) Methods

Assigning value one by one

```
class Employee
{
    eid: number;
    ename: string;
    deptno: number;
    display(): void
    {
        Console.log(this.eid);
        Console.log(this.ename);
        Console.log(this.deptno);
    }
}
var emp = new Employee();
emp.eid = 1;
emp.ename = "A";
emp.deptno = 10;
emp.display();
```

Assigning values by method.

```
setData(eid: number, ename: string, deptno: number)
{
    this.eid = eid;
    this.ename = ename;
    this.deptno = deptno;
}
emp.setData(101, "John", 10);
emp.display();
```

Constructor injection:

```
Employee Constructor(eid: number, ename: string, deptno: number)
{
    this.eid = eid;
    this.ename = ename;
    this.deptno = deptno.
}
var Employee emp = new Employee(1, "A", 2);
display();
```

Inheritance:

```
class person
{
    name:string;
    constructor(name:string)
    {
        this.name=name;
    }
}
class employee extends person
{
    empno:number;
    constructor(empno:number, name:string)
    {
        super(name);
        this.empno = empno;
    }
    display():void
    {
        console.log(this.empno);
        console.log(this.name);
    }
}
var employee = new employee(1, "John");
emp.display();
```

Ex:2: Overriding Concept

```
class Bank
{
    roi:number=0;
    display():number
    {
        return this.roi;
    }
}
class BankX extends Bank
{
    display():number
    {
        return 10.5;
    }
}
class BankY extends Bank
{
    display():number
    {
        return 20;
    }
}
var b = new BankX();
console.log(b.display());
var
```

```
var x = new BankX();
console.log(x.display());  //10.5
var y = new BankY()
console.log(y.display());  //20
```

Interface in Typescript:

→ interface is a structure that defines the contract in your application.

→ It defines the syntax for classes to follow.

→ The Typescript compiler does not convert interface to javascript. It uses interface for type checking. This is also known as "duck typing" or "structural subtyping".

→ An interface is defined with the keyword interface and it can include properties and method declarations using a function or an arrow function.

→ An interface extends another interface

→ A class implements interface.

ex1:

```
interface IEmployee
{
    empName: string;
    empID: number;
    empSal: number;
    dispData: () => void; -> Un-implemented fn()
}

var emp: IEmployee = { empName = "John",
                       empID = 101,
                       empSal = 50000,
                       dispData(): void {
                       Console.log(this.empName+" "+this.empID
                                                  +" "+this.empSal);
                       }

Console.log(emp.empName);
Console.log(emp.empID);
Console.log(emp.empSal);

emp.dispData();
```

o/p:

```
John
101
50000
John  101  50000
```

Ex:2

```
interface I1
{
    a:number;
    b:number;
    Sum():number;
}
interface I2 extends I1
{
    x:number;
    y:number;
    Sub():number;
}
class C1 implements I2
{
    a:number;
    b:number;
    x:number;
    y:number;
    Sum():number
    {
    return(this.a+this.b);
    }
    Sub():number
    {
    return(this.x-this.y);
    }
var c=new C1();
    c.a=100;
    c.b=200;
    c.x=2000;
    c.y=3000;
console.log(Sum());// 3000
Console.log(Sub());// -1000
```

Need to redefine values in the child class

Modules in Typescript

→ The files created in typescript have global access, which means that variables declared in one file are easily accessed in other file.

→ This global nature can cause code conflicts can cause issues with execution at runtime.

→ we have export and import module functionality which can be used to avoid global variables function conflicts.

→ This feature is available in Javascript with ES6 release and also supported in typescript.

Test1.ts → Global Scope → Test2.ts

```
//variable
var x:string = "welcome";
//simple function
function myfunc():void
{
Console.log("This is my function");
}
//class
class myclass
{
  a:number;
  b:number;
  constructor(a:number,b:number)
  {
    this.a=a;
    this.b=b;
  }
  add=()=>
  {
    return (this.a+this.b);
  }
```

```
console.log(x);
myfunc();
var c=new myclass(10,20);
Console.log(c.add());
```

Export & Import:

→ Export is used for accessing the variables and function and others within the same file. (test1.ts)

→ Import is used for accessing the variables, methods, functions of other file (test1.ts) from other file (test2.ts).

→ For compilation we use the cmd as

tsc --module commonjs ./test1.ts

tsc --module commonjs ./test2.ts.

For executing we follow the normal commands.

```
node ./test1.js
node ./test2.js
```

test1.ts

```
//variable
export var x:string="welcome";
// simple function
export function myfunc():void
{
console.log("This is simple function");
}
// class
export class myclass
{ a:number
  b:number;
constructor(a:number,b:number)
{ this.a=a;
  this.b=b;
}
add=()=>
{ return(this.a+this.b);
}
```

test2.ts

```
import{x} from "./test1.ts"
  console.log(x);
import{myfunc} from "./test1.ts"
import{myclass} from "./test1.ts"
var c=new myclass(10,20);
console.log(c.add());
```